# MINI PROJECT REPORT

**Foundations of Programming – CSE10010**

## Strategic Tic Tac Toe

*Built in C Using Raylib*


**Team Members:**

Akanksha Dange – 1262251158

Ishita Kale – 1262251255

Eshita Uchgaonkar – 1262252018

Anvesha Jain – 1262253117

Adhitte Nair – 1262253453

Ananya Bhale – 1262254062

MIT World Peace University, Pune

# Table of Contents

1. Title Page ..... 0

2. Acknowledgement ..... 2

3. Abstract ..... 3

4. Introduction ..... 4

5. Methodology ..... 6

6. Implementation ..... 9

7. Results and Discussion ..... 17

8. Conclusion ..... 19

9. Future Scope ..... 20

## Acknowledgment

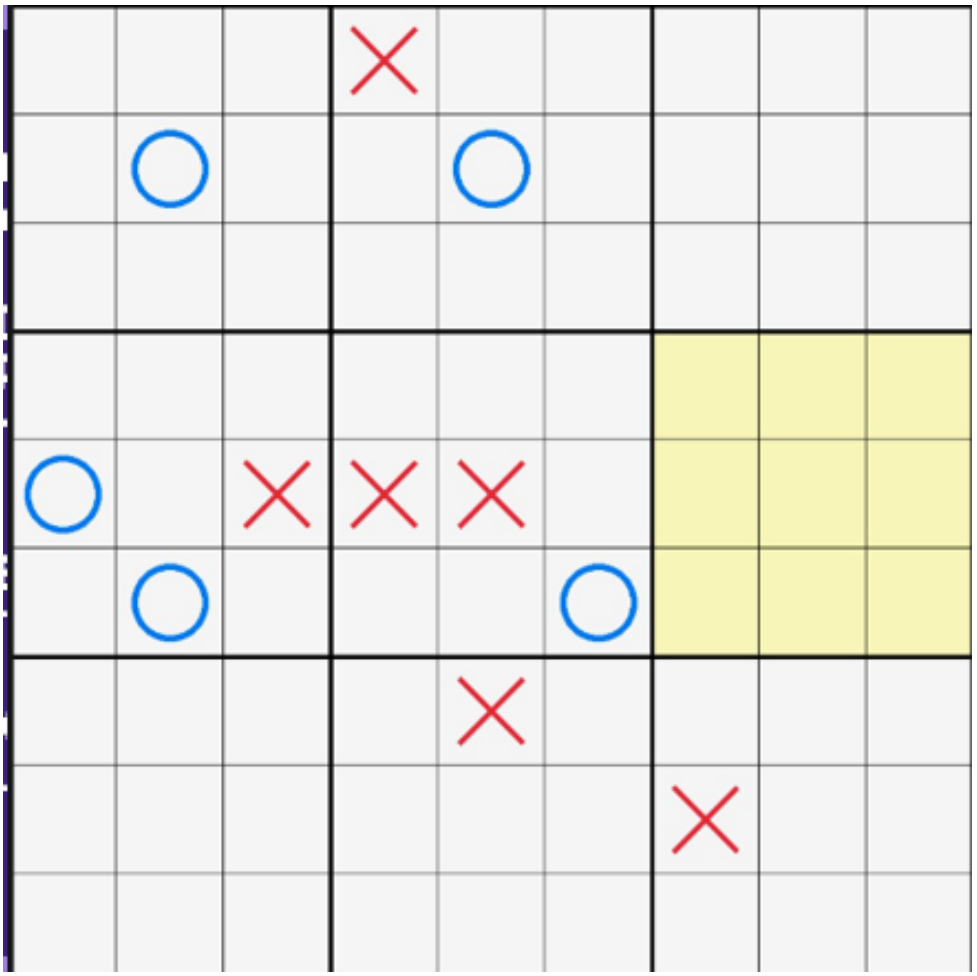
We would like to express our sincere gratitude to our project guide and faculty of Foundations of Programming course (CSE10010), Dr. Renuka Mane at MIT World Peace University, Pune, for her invaluable guidance, encouragement, and support throughout the development of this project.

We are also thankful to our peers and teammates for their collaborative spirit and constructive feedback that helped us improve the game at every stage. Special thanks to the Raylib open source community whose documentation and resources greatly aided our implementation.

Lastly, we acknowledge all those who directly or indirectly contributed to making this project a success.

## Abstract

Strategic Tic-Tac-Toe is a board game that has nine tic-tac-toe boards arranged in a 3x3 grid.The game is played on the tic-tac-toe boards until one of them wins on the Ultimate Tic-Tac-Toe game.Compared to other tic-tac-toe games,strategy is conceptually more complex and has been found to be more complex for computers.Additional features include Misère Mode,a one-time swap system,sound effects,and better menu and instructions screens.The project is an example of the practical application of programming concepts such as 2D arrays,functions,loops,and conditional statements and is also an example of the better understanding of logical thinking and programming concepts that the team has acquired.



# 1. Introduction

**Problem Statement**

Traditional Tic Tac Toe is a simple game that becomes boring once the basic strategy is understood. There exists a need for a more engaging, strategy intensive variation that challenges players to think multiple moves ahead while still being accessible to new players. This project address that need by implementing strategic Tic Tac Toe - a nested board variant variant that increases strategic depth while retaining almost the same rules

**Background**

The two players (X and O) alternate, just like in standard tic tac toe. At the beginning of the game, either X or O can play in any of the 81 available spots. Every subsequent move requires the players to play in the area of the larger board that corresponds to the location of the previous move within the smaller board.

The player in the larger board marks the entire small board as won if a move is made that, according to standard tic tac toe rules, wins a small board. No more moves can be made in a small board once it has been won by a player or filled to capacity, and a player is sent to that board.

The player in the larger board marks the entire small board as won if a move is made that, according to standard tic tac toe rules, wins a small board. A player sent to a small board may select any other legitimate board to play on after the board is filled or won by a player.

Either a player wins the larger board or there are no more playable small boards, in which case the game is a draw.This project uses C as the main programming language, GCC for compilation, and Raylib, a straightforward, user-friendly library for enjoying video game programming, for rendering.

**Objectives**

- To implement a fully functional strategic tic tac toe game using C and Raylib.

- To incorporate a nested board structure(9 small boards within one large board).
- To implement the forced move mechanic
- To add a misère game mode where the winning condition is reversed.
- To implement a swap system as a one time strategic ability.
- To enhance the user experience by adding sound effects

## 2. System Design / Methodology

### Synopsis of the Method

Raylib's window and input management power the game's central game loop. For small boards, the game state is represented by a four-dimensional array; for large boards, it is represented by a two-dimensional array. The MENU, PLAYING, SWAP, and GAMEOVER states are all managed by the program.

### Board Structure

| Data Structure | Description |
|---|---|
| sb[3][3][3][3] | Four-dimensional array representing 9 small boards, each containing a 3×3 grid of cells. |
| bb[3][3] | Two-dimensional array representing the state of the large board (which player has claimed which small board). |
| freeMove flag | Boolean flag indicating whether the current player may choose any available small board. |
| forcedBoard | Coordinates of the small board in which the current player must play. |

**The Flow of the Game**

Three crucial phases can be used to characterize the overall game algorithm:

**1- Setup and Initialization**

The program initializes all arrays and configures flags and variables when the game launches. All of the small boards and the large board are empty. The forced board pointers are reset, the game state is changed to MENU, and players are given their marks (X and O).

**2 - Main Game Loop**

The Raylib window stays open while the game runs continuously. In every iteration:

• The current state of the game is assessed (MENU, PLAYING, SWAP, GAMEOVER).

• The player's move updates the board state.

• After each move, the conditions for winning and drawing are assessed.

• Every frame, the screen is redrawn.

**3 - Turn Logic**

When a mouse click is detected in the playing state:

- The target small board and cell position are determined from the mouse coordinates.
- The cell must be empty and the small board must be the forced board(or free move must be enabled)
- The player's mark is placed in the cell.
- The small board is checked for a win or draw; if won, the large board is updated.
- The next forced board is set based on the cell position just played.
- If the target forced board is already won or drawn, freeMove is enabled.
- The large board is checked for an overall win (Normal or Misère mode) or draw.

- If the game is not over, the turn switches to the other player.

**Swap System**

Each player is allowed one swap action per game.When a player wins a small board and the swap has not yet been used, the game enters the SWAP state.The player selects another small board on the large board; the contents of the two small boards are exchanged.

**Misère Mode**

This mode is similar to the Normal mode. The only difference is that in the Misere Mode is that the player who occupies three boxes in a row loses the game.

## 3. Implementation

### Development Environment

The project was developed on Windows using Visual Studio Code as the code editor and GCC as the compiler. The Raylib library was integrated for all graphical rendering, input handling, audio playback, and window management. The game is compiled and run as a native Windows application.

### Key Algorithms and Code Logic

### Board Initialization

At the start of each game, all cells in SB[3][3][3][3] are set to EMPTY, BB[3][3] is cleared, player turn is set to PLAYER_X, freeMove is set to false, and swapUsed is set to false.

### Move Validation and Placement

When a player clicks on the board, the program maps the mouse coordinates to a small board index (i, j) and a cell index (k, l). The move is valid only if: the cell sb[i][j][k][l] is EMPTY, and either freeMove is true or (i, j) matches the current forcedBoard. If valid, the cell is updated with the current player's mark.

### Win Detection

After each move, the program checks three winning line patterns for the relevant board: all three cells in a row, all three cells in a column, and the two diagonal lines. This logic is applied first to the small board to determine if a player has claimed it, and then again to the large board to determine the game winner.

### Sound Effects

On the basis of project recommendations, sound feedback was added as an improvement. A small board win, a small board loss or draw, and the final game over (win, loss, or draw) sound effects were taken from Kenney.nl. During playtesting, this greatly enhanced the user experience.

**Swap State Execution**

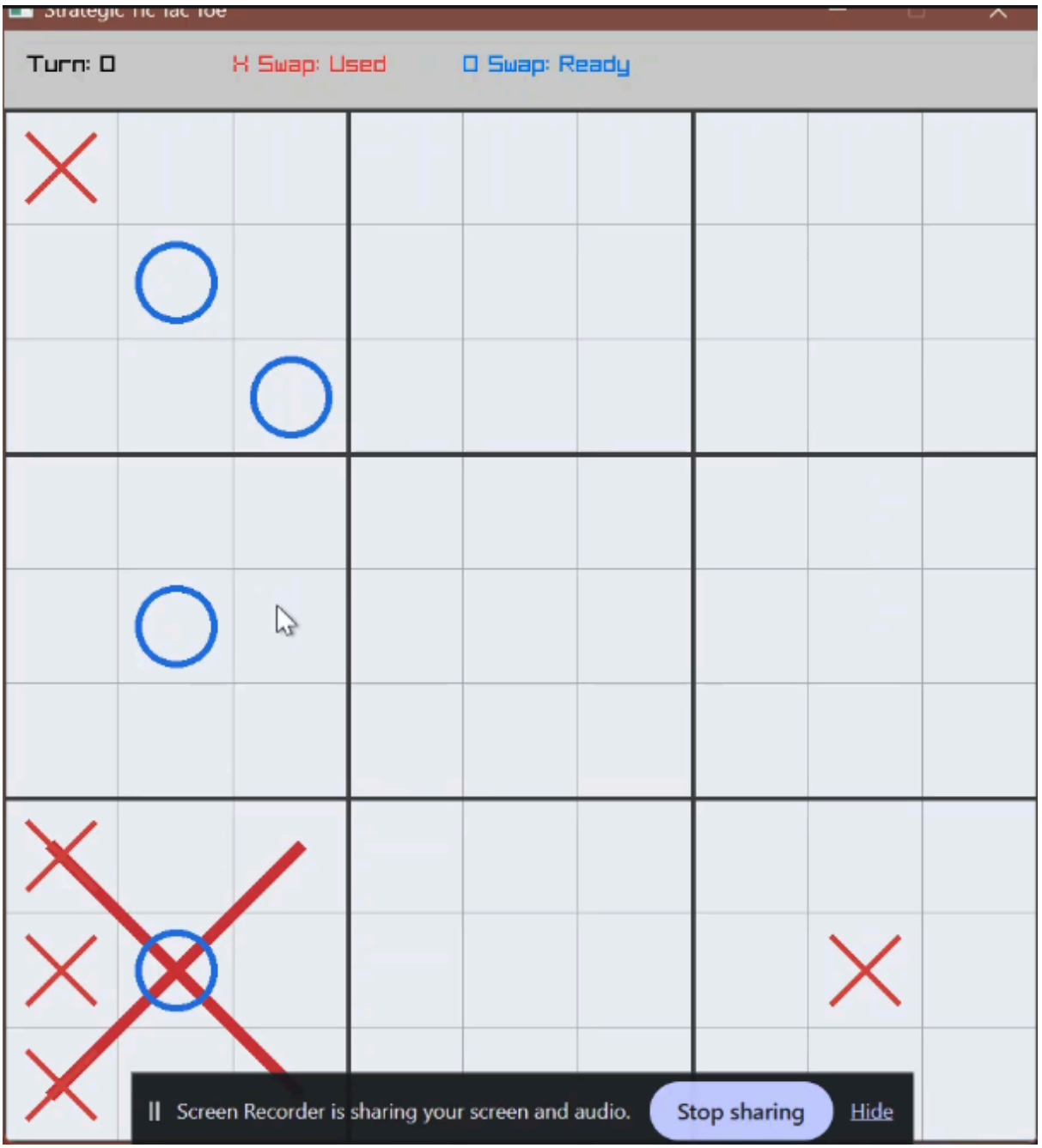
The large board visually indicates the small boards that are available when the SWAP state is reached. The game switches the winner entry in BB and all cell entries in SB for the two boards when the player clicks on a valid board. Following the swap, freeMove is activated, swapUsed is set to true, the state reverts to PLAYING, and the opponent takes the turn.

**Testing**

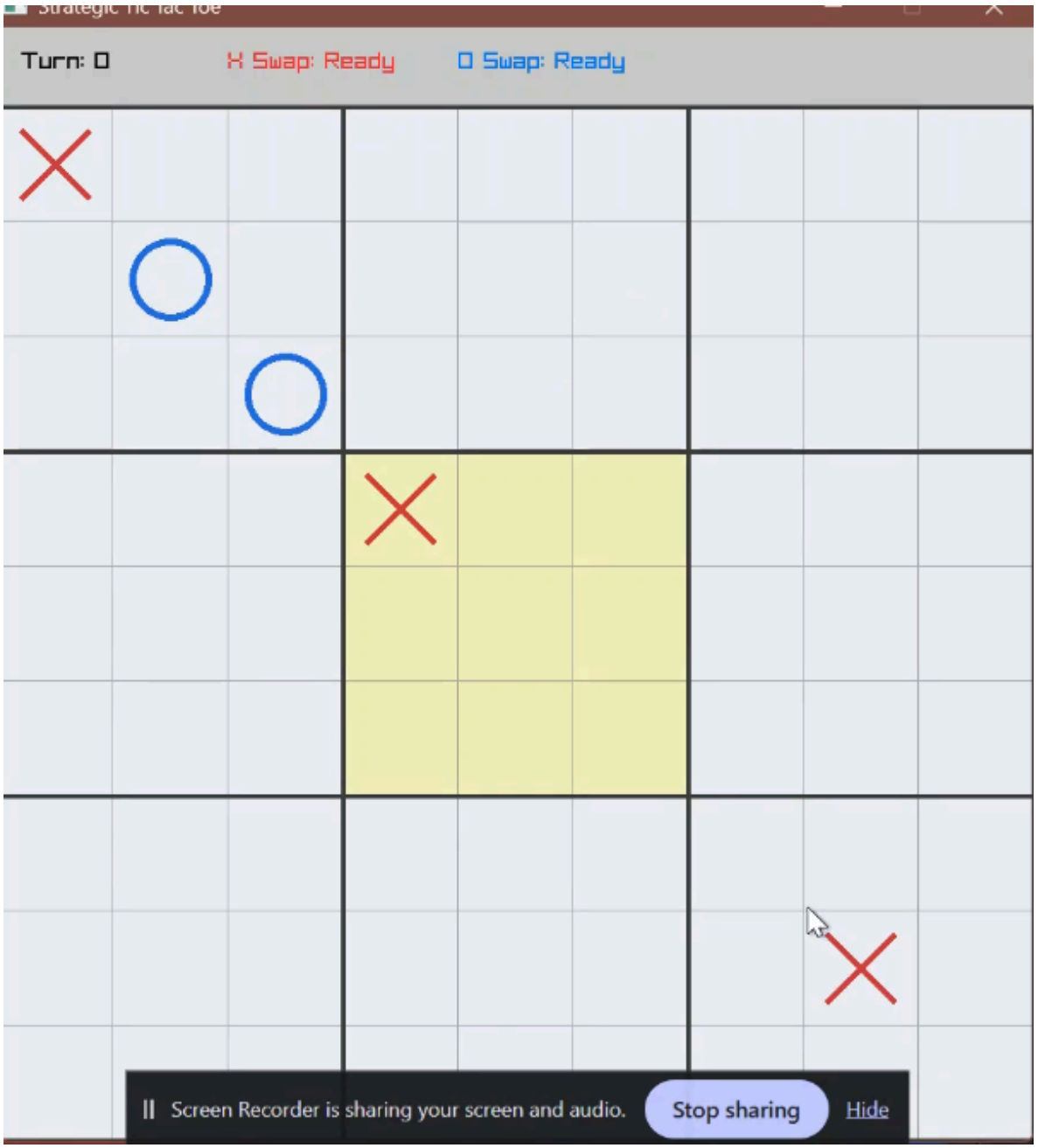
| No. | Test Case | Expected Output | Result |
|---|---|---|---|
| 1 | Player wins a small board | BB updated; forced board set correctly | Pass |
| 2 | Forced board already won – freeMove | freeMove enabled; player can pick any board | Pass |
| 3 | Normal mode: player wins large board | Winner declared; GAMEOVER state entered | Pass |
| 4 | Misère mode: player gets 3 in a row | That player loses; opponent declared winner | Pass |

| | | | |
|---|---|---|---|
| 5 | All boards drawn – overall draw | Draw declared | Pass |

Main Menu

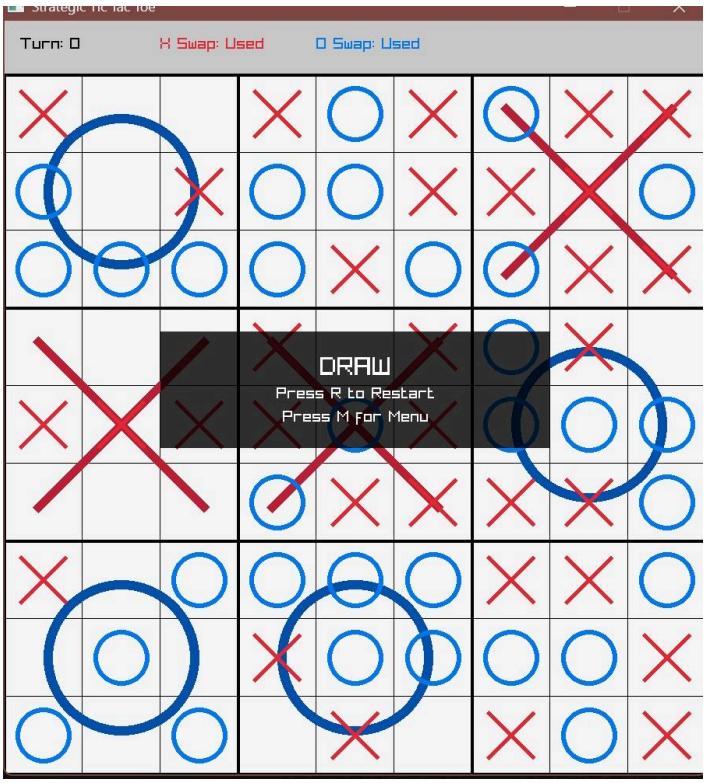

Turn: O
X Swap: Used
O Swap: Ready
Screen Recorder is sharing your screen and audio.
Stop sharing
Hide

Freemove




Forced board highlighted in yellow

Small board win, ready to swap




Draw

Player O wins

Win in Misere mode

## 4. Results and Discussion

### Output

Using Raylib, the program successfully opens a graphical window. Players can choose between Normal mode and Misère mode when the main menu appears upon startup. After choosing a mode, the game board is displayed with all nine embedded small boards and a large 3x3 grid.

While playing:

• The current player's symbol (X or O) is placed in the chosen cell with each legitimate move.

• The forced small board for the subsequent turn is visually highlighted, and the board updates instantly following each move.

• The winning player's symbol is displayed in the large board cell for a small board that is won.

• Moves, wins, and draws are accompanied by sound effects.

• A popup that explains the swap mechanism and asks the player to choose a board appears when they try to swap.

• The outcome is shown at the conclusion of the game.

• The player can press M to go back to the main menu or R to restart.

## Analysis

Every goal was effectively accomplished. As intended, the nested board structure requires players to consider not only winning the current small board but also which small board they are sending their opponent to. Compared to traditional Tic Tac Toe, this results in a much more strategic and captivating gaming experience.

In all test runs, the player who finished a row of three won small boards on the large board was declared the loser, in accordance with the reversed winning condition, demonstrating that the Misère mode was correctly implemented.

The Swap system was observed to significantly change the outcome of test games, confirming the mechanic's strategic value.

The improvements made based on Dr. Renuka’s recommendations — sound effects, popup guidance for swap, improved menu, and clearer instructions  were implemented successfully.

## 5. Conclusion

This project used the Raylib library to successfully implement Strategic Tic Tac Toe in C. With a nested board structure, a forced move mechanism, a Misère game mode, a one-time Swap ability, and audio-visual improvements, the final product is a fully playable, graphically rendered two-player application. Because controlling the state interactions between the nested boards, the forced move system, and the swap mechanic required careful logical design, extensive testing and debugging. Overall, all goals were met, and the finished product shows how programming principles can be applied coherently in a fun and strategic gaming environment.

## 6. Future Scope

While the current implementation meets all project objectives, several enhancements could be explored in future iterations:

- AI Opponent: Implement a computer-controlled player using algorithms such as Minimax to allow gameplay against the machine.
- Score Tracking System: Add a persistent score tracker that records wins, losses, and draws across multiple sessions
- Online Multiplayer: Extend the game to support online two-player matches over a network using socket programming or a backend service.